THE UNIVERSITY OF AZAD

JAMMU & KASHMIR

Department of Software Engineering

IPT

Lab task no 03

Instructor Name: Engr. Sidra Rafique

Semester: 5th

Session: 2024 – 28

Name: Raja Ali Shan

Roll No: 2024 – SET – 18

LAB 03 (IPT)

Task 1: NumPy array, sum and average

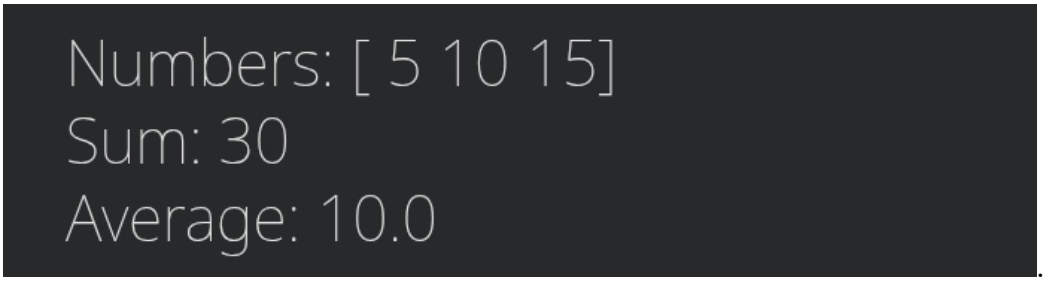
CODE

```
import numpy as np

numbers = np.array([5, 10, 15])

print("Numbers:", numbers)
print("Sum:",
np.sum(numbers))
print("Average:",
np.mean(numbers))
```

OUTPUT



.0

Task 2: Write your own text on a black image

CODE

```
import cv2 import numpy as np
from google.colab.patches import
cv2_imshow

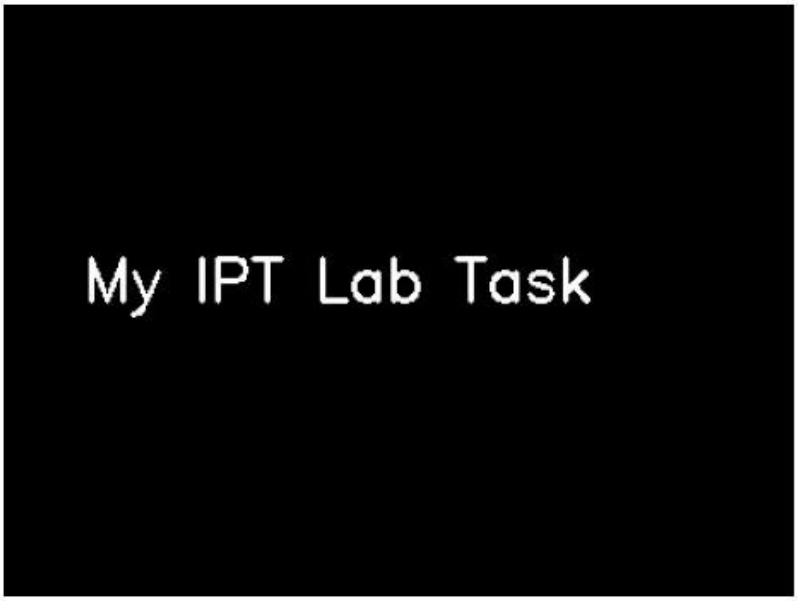
image = np.zeros((300, 400, 3), dtype=np.uint8)

cv2.putText(
image,    "My IPT
Lab Task",    (40,
150),
cv2.FONT_HERSH
EY_SIMPLEX,    1,
   (255, 255, 255),

2
)

cv2_imshow(image)
```

OUTPUT



Task 3: Change image dimensions

CODE

```
import cv2 import numpy as np
from google.colab.patches import
cv2_imshow

image = np.zeros((200, 500, 3), dtype=np.uint8)

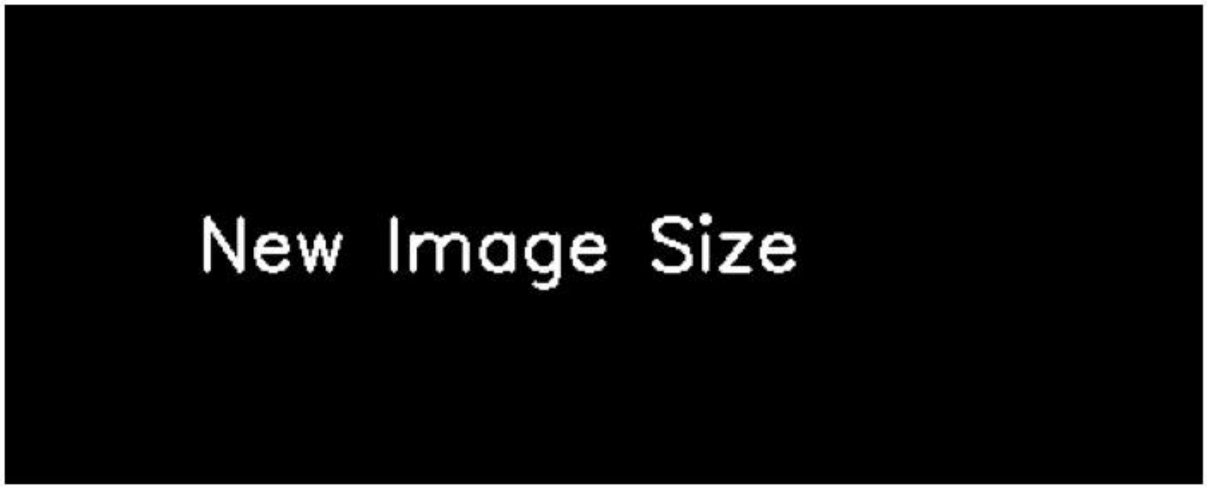
cv2.putText(
image,    "New
Image Size",    (80,
110),
cv2.FONT_HERSH
EY_SIMPLEX,    1,
   (255, 255, 255),

2
)

print("Image Shape:",
image.shape)
cv2_imshow(image)
```

OUTPUT

```
Image Shape: (200, 500, 3)
```



Task 4: Red text using BGR colour

CODE

```
import cv2 import numpy as np
from google.colab.patches import
cv2_imshow

image = np.zeros((300, 400, 3), dtype=np.uint8)

cv2.putText(
image,    "Red
Color Text",    (40,
150),
cv2.FONT_HERSH
EY_SIMPLEX,    1,
   (0, 0, 255),

2
)

cv2_imshow(image)
```

OUTPUT



Image Processing Task 1: Load and display an image

CODE

```
import cv2 import
numpy as np import
matplotlib.pyplot as
plt from skimage
import data

# Load sample
RGB image img =
data.astronaut()

# Display image information
print("Image Shape:",
img.shape) print("Image Data
Type:", img.dtype)

# Display image
plt.imshow(img)
plt.title("Original
Image")
plt.axis("off")
plt.show()
```

OUTPUT



Image Processing Task 2: Display RGB components

CODE

```
import cv2 import
numpy as np import
matplotlib.pyplot as
plt from skimage
import data

# Load RGB
image img =
data.astronaut(
)

# Split RGB channels
R, G, B = cv2.split(img)

# Display RGB components fig, ax
= plt.subplots(1, 3, figsize=(10, 3))

ax[0].imshow(R, cmap="Reds")
ax[0].set_title("Red Channel")

ax[1].imshow(G,
cmap="Greens")
ax[1].set_title("Green Channel")

ax[2].imshow(B, cmap="Blues")
ax[2].set_title("Blue Channel")
```

```
for a in ax:
    a.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT



Image Processing Task 3: Convert image to grayscale

CODE

```
import cv2 import
numpy as np import
matplotlib.pyplot as
plt from skimage
import data

# Load RGB
image img =
data.astronaut(
)

# Convert RGB to grayscale gray
= cv2.cvtColor(img,
cv2.COLOR_RGB2GRAY)

# Display image information
print("Grayscale Shape:",
gray.shape) print("Data Type:",
gray.dtype)

# Display grayscale
image
plt.imshow(gray,
cmap="gray")
plt.title("Grayscale
Image")
```

```
plt.axis("off")
plt.show()
```

OUTPUT



Image Processing Task 4: Global and adaptive thresholding

CODE

```
import cv2 import
numpy as np import
matplotlib.pyplot as
plt from skimage
import data

# Load image and convert to
grayscale img = data.astronaut()
gray = cv2.cvtColor(img,
cv2.COLOR_RGB2GRAY)

# Apply global thresholding
_, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

# Apply adaptive
thresholding adaptive =
cv2.adaptiveThreshold(
gray,     255,
cv2.ADAPTIVE_THRES
H_GAUSSIAN_C,
cv2.THRESH_BINARY,
31,
   5
)
```

```
# Display both results fig, ax =
plt.subplots(1, 2, figsize=(8, 3))

ax[0].imshow(binary,
cmap="gray")
ax[0].set_title("Global
Thresholding")

ax[1].imshow(adaptive,
cmap="gray")
ax[1].set_title("Adaptive
Thresholding")

for a in ax:
    a.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT

